

## Building Predictive Model for Gas Energy Demand

Jack Malik

June 9, 2020

## Table of Content

|                                                                |          |
|----------------------------------------------------------------|----------|
| <b>1. PROBLEM STATEMENT .....</b>                              | <b>3</b> |
| <b>2. IMPLEMENTATION APPROACH .....</b>                        | <b>3</b> |
| 2.1. Data Description .....                                    | 3        |
| 2.2. Building a Correlation Matrix of Features and Target..... | 4        |
| <b>3. BUILDING MODEL.....</b>                                  | <b>5</b> |
| <b>4. COMPUTING COMMON MODEL ERRORS.....</b>                   | <b>6</b> |
| <b>5. PROGRAM OUTPUT.....</b>                                  | <b>6</b> |
| <b>6. APPENDIX A – ENERGY_PREDICTOR.PY SOURCE FILE.....</b>    | <b>7</b> |

# 1. Problem Statement

A data file in a csv format contains a daily timeseries of historic weather observations (features), also called predictors and gas demand actuals (target) for gas years 2013 to 2018 inclusive for a fictional country. The objective is to use the Python toolset to derive a forecasting model that returns out of sample predictions for gas year 2018.

## 2. Implementation Approach

We observe 4 predictors in the data file and a single target. Our strategy depends on the results of our *Exploratory Data Analysis* (EDS). We analyze all predictors and the level of correlation between them as well as between the target.

The selection of our predictive model depends on our understanding of data achieved during the EDS process we perform and the conclusions we draw which model could be most effective in predicting the target.

We first build the correlation matrix of all features and target. We hope to eliminate predictors if they are correlated to simplify the process.

### 2.1. Data Description

We use Python ***describe()*** function to view data characteristics. The output of the function looks as follows:

|       | station_01_tem. | station_02_tem. | station_01_wind | station_02_wind | gas_demand_volume |
|-------|-----------------|-----------------|-----------------|-----------------|-------------------|
| count | 2165            | 2164            | 2164            | 2163            | 2165              |
| mean  | 11.56079908     | 11.08987985     | 8.307989834     | 6.532556634     | 89.36482217       |
| std   | 6.48945676      | 6.51711694      | 3.220903693     | 2.510031544     | 51.07682739       |
| min   | -7.3            | -6.07           | 2.53            | 1.84            | 24.64             |
| 25%   | 6.24            | 5.7375          | 6.03            | 4.76            | 41.61             |
| 50%   | 11.06           | 10.93           | 7.695           | 6.09            | 76.64             |
| 75%   | 16.97           | 16.39           | 9.9125          | 7.7             | 131.08            |
| max   | 29.21           | 28.13           | 22.19           | 18.87           | 270.69            |

## 2.2. Building a Correlation Matrix of Features and Target

We build a correlation matrix of features and target to possibly eliminate features which are correlated between themselves in hope to limit overall number of features. The program builds the following full correlation matrix:

Full correlation matrix of all features:

|                  | station_01_temp. | station_02_temp. | station_01_wind | station_02_wind | gas_demand_vol. |
|------------------|------------------|------------------|-----------------|-----------------|-----------------|
| station_01_temp. | 1                | 0.98997923       | -0.205899783    | -0.155647172    | -0.940276696    |
| station_02_temp. | 0.98997923       | 1                | -0.215077823    | -0.143630781    | -0.937628986    |
| station_01_wind  | -0.205899783     | -0.215077823     | 1               | 0.910181372     | 0.257729078     |
| station_02_wind  | -0.155647172     | -0.143630781     | 0.910181372     | 1               | 0.217591006     |
| gas_demand_vol.  | -0.940276696     | -0.937628986     | 0.257729078     | 0.217591006     | 1               |

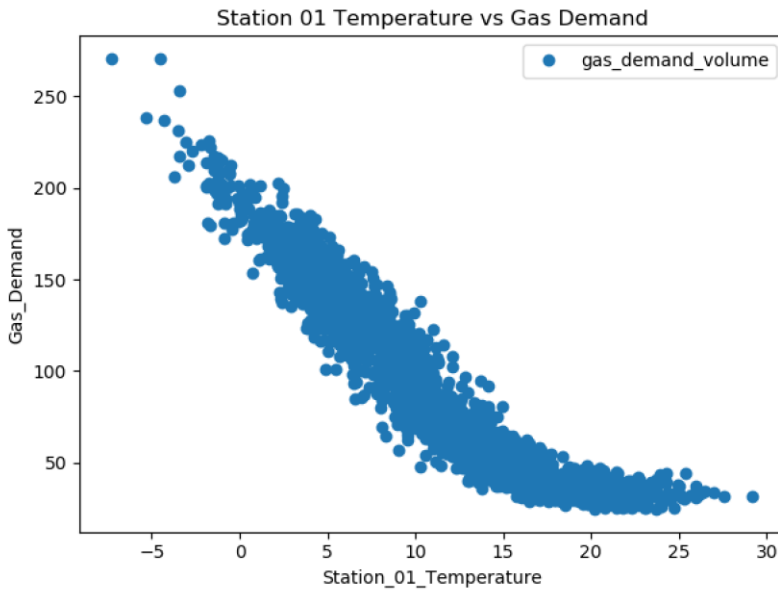
We observe strong correlation between '**station\_01\_temperature**' and '**station\_02\_temperature**' predictors while noticing that they are both very correlated with the volume of gas demand.

We also observe no correlation between temperature of any of the 2 stations and the levels of wind reported in those stations. Moreover, the levels of winds are not correlated with gas demand.

The above observations allow us to remove one of the station temperatures as well as winds recordings in both stations as not affecting gas demand volumes and therefore insignificant in predicting the target value of gas demand volumes.

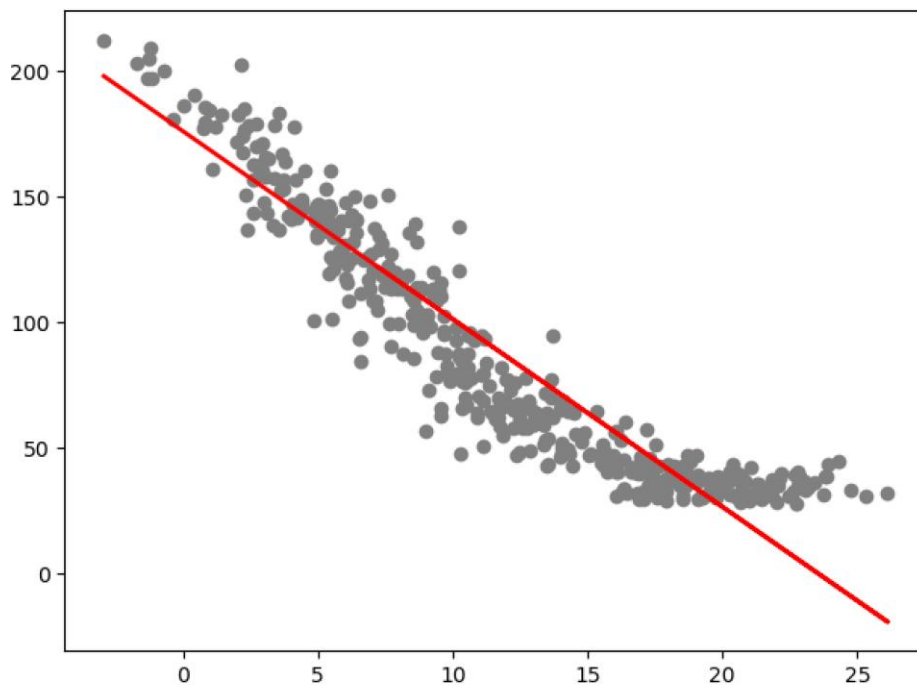
We are therefore left with a single predictor '**station\_01\_temperature**' and a target data stored in '**gas\_demand\_vol**'. Our reduced correlation matrix looks as follows:

|                   | station_01_temp. | station_01_wind | gas_demand_volume |
|-------------------|------------------|-----------------|-------------------|
| station_01_temp.  | 1                | -0.205899783    | -0.940276696      |
| station_01_wind   | -0.205899783     | 1               | 0.257729078       |
| gas_demand_volume | -0.940276696     | 0.257729078     | 1                 |



### 3. Building Model

We observe strong correlation between 'station\_01\_temperature' and 'gas\_demand\_volume'. We therefore decide to build a simple univariate linear regression model to predict gas demand for year 2018. Our linear regression model can be visualized as follows:



## 4. Computing Common Model Errors

Mean Absolute Error: 15.901675177380474

Mean Squared Error: 416.7833149689213

Root Mean Squared Error: 20.415271611441305

## 5. Program Output

Below, we show a partial content of the final Excel file the program produces. The file contains 3 columns: Date, Actual Demand and Predicted Demand for gas during the month of July, 2018.

|    | Date                | Actual Demand | Predicted Demand |
|----|---------------------|---------------|------------------|
| 0  | 2018-07-01 00:00:00 | 30.34         | 28.60556262      |
| 1  | 2018-07-02 00:00:00 | 37.91         | 25.07608093      |
| 2  | 2018-07-03 00:00:00 | 38.38         | 20.3956813       |
| 3  | 2018-07-04 00:00:00 | 37.81         | 9.96069195       |
| 4  | 2018-07-05 00:00:00 | 38.35         | 12.18579997      |
| 5  | 2018-07-06 00:00:00 | 34.38         | 23.46479581      |
| 6  | 2018-07-07 00:00:00 | 29.1          | 18.78439618      |
| 7  | 2018-07-08 00:00:00 | 28.11         | 23.46479581      |
| 8  | 2018-07-09 00:00:00 | 36.58         | 24.84589734      |
| 9  | 2018-07-10 00:00:00 | 39.5          | 48.17116765      |
| 10 | 2018-07-11 00:00:00 | 38.57         | 39.27073556      |
| 11 | 2018-07-12 00:00:00 | 38.3          | 30.67721492      |
| 12 | 2018-07-13 00:00:00 | 35.87         | 26.61063819      |
| 13 | 2018-07-14 00:00:00 | 29.2          | 19.16803549      |

..... more data

When executed the program creates 4 different Excel files:

- 1/. Output of the describe() function.
- 2/. Full correlation matrix
- 3/. Stripped correlation matrix
- 4/. File containing final output of actual and predicted gas demand volumes as shown above.

## 6. Final Remarks

The program has been developed using Anaconda 2018.12 – built py37\_0. Python version 3.7 was used. The program makes no attempt to take any user input or test code through any Unit Testing.

The program assumes that the given input data file name is 'weather\_and\_gas\_demand\_historic\_actuals.csv' and it is located in the same directory as the actual 'energy\_predictor.py' Python file. The source code is given below.

## 7. Appendix A – energy\_predictor.py Source File

```
import logging
import sys
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import metrics
import scipy as sp
from scipy.stats import shapiro

logger_ = logging.getLogger(__name__)
logger_.setLevel(logging.INFO)
handler = logging.StreamHandler(sys.stdout)
handler.setLevel(logging.INFO)
formatter = logging.Formatter('%(asctime)s)-[%(name)s]-[%(levelname)s]: %(message)s')
handler.setFormatter(formatter)
logger_.addHandler(handler)

class EnergyFeaturesCorrelationMatrix(object):

    @staticmethod
    def get_energy_tm_data(full_path_file_name):
        data_frame = pd.read_csv(full_path_file_name)
        return data_frame

    @staticmethod
    def build(src_file_name):

        """
        Parsing energy demand file given. The content of the form:
```

```
<dy>,<station_01_temperature_degrees_celcius>,<station_02_temperature_degrees_celcius>,  
<station_01_wind_speed_m_per_s>,<station_02_wind_speed_m_per_s>,<gas_demand_mcm>
```

```
:param src_file_name: source file given as part of exercise
```

```
:return: Pandas data frame containing file data
```

```
"""
```

```
try:
```

```
    df = EnergyFeaturesCorrelationMatrix.get_energy_tm_data(src_file_name)
```

```
    df.dropna()
```

```
    df['dy'] = pd.to_datetime(df['dy'], format='%d/%m/%Y')
```

```
    df.columns = ['date',
```

```
                  'station_01_temperature',
```

```
                  'station_02_temperature',
```

```
                  'station_01_wind',
```

```
                  'station_02_wind',
```

```
                  'gas_demand_volume']
```

```
    df = df[pd.to_numeric(df['gas_demand_volume'], errors='coerce').notnull()]
```

```
    df = df[pd.to_numeric(df['station_01_temperature'], errors='coerce').notnull()]
```

```
    with pd.ExcelWriter('weather_and_gas_demand_historic_actuals_cleaned.xlsx') as writer:
```

```
        df.to_excel(writer, sheet_name='Cleaned-Data')
```

```
    return df
```

```
except Exception as ex:
```

```
    logger_.error("Failed. Msg: {}".format(ex))
```

```
    return None
```

```
def main(argv):
```

```
    logger_.info("Starting program .. ")
```

```
    # create correlation matrix of all energy demand features
```

```
    mtx = EnergyFeaturesCorrelationMatrix()
```

```
    data_frame = mtx.build("weather_and_gas_demand_historic_actuals.csv")
```

```
    if data_frame is None:
```

```
        logger_.error("Unexpected exception while building data frame.")
```

```
        return 1
```

```
    data_frame.plot(x='station_01_temperature', y='gas_demand_volume', style='o')
```

```
    plt.title('Station 01 Temperature vs Gas Demand')
```

```
    plt.xlabel('Station_01_Temperature')
```

```
    plt.ylabel('Gas_Demand')
```

```
    plt.show()
```

```
    plt.close()
```

```
    data_frame_no_date = data_frame.drop(['date'], axis=1, inplace=False)
```

```

corr_matrix_full = data_frame_no_date.corr()
print(corr_matrix_full)
with pd.ExcelWriter('correlation_matrix_full.xlsx') as writer:
    corr_matrix_full.to_excel(writer, sheet_name='CORR-VER-1')

# remove unnecessary columns
corr_matrix_stripped = corr_matrix_full.drop(['station_02_temperature',
                                             'station_02_wind'],
                                             axis=1, inplace=False)
# remove unnecessary rows
corr_matrix_stripped.drop(['station_02_temperature', 'station_02_wind'], inplace=True)
with pd.ExcelWriter('correlation_matrix_stripped.xlsx') as writer:
    corr_matrix_stripped.to_excel(writer, sheet_name='CORR-VER-2')

# -----
# stripped correlation matrix of the station 2 parameters as correlated
# -----
#
#          station_01_temperature  station_01_wind  gas_demand
# station_01_temperature  1          -0.205899783    -0.940276696
# station_01_wind        -0.205899783          1         0.257729078
# gas_demand             -0.940276696        0.257729078         1
# -----

# Now remove station_01_wind column and row from correlation matrix
corr_matrix = corr_matrix_stripped.drop(['station_01_wind'], inplace=False)
corr_matrix.drop(['station_01_wind'], axis=1, inplace=True)
with pd.ExcelWriter('correlation_matrix.xlsx') as writer:
    corr_matrix.to_excel(writer, sheet_name='CORR-MTX-FINAL')

# Perform test for normality of distribution of station_01_temperature values
distance, p_value = sp.stats.normaltest(data_frame['gas_demand_volume'])
stat, s_p_value = shapiro(data_frame['gas_demand_volume'])

x = data_frame['station_01_temperature'].values.reshape(-1,1)
y = data_frame['gas_demand_volume'].values.reshape(-1,1)
dates = data_frame['date'].values.reshape(-1,1)

x_train, x_test, y_train, y_test = train_test_split(x, y,
                                                    test_size=0.2105,
                                                    shuffle=False)

x_train_, x_test_, dates_train, dates_test = train_test_split(x, dates,
                                                            test_size=0.2105,
                                                            shuffle=False)

regressor = LinearRegression()

```

```

# training the algorithm
regressor.fit(x_train, y_train)
# Predicting
y_pred = regressor.predict(x_test)

plt.clf()
plt.scatter(x_test, y_test, color='gray')
plt.plot(x_test, y_pred, color='red', linewidth=2)
plt.show()

description = data_frame.describe()
with pd.ExcelWriter('description.xlsx') as writer:
    description.to_excel(writer, sheet_name='Info')

logger_.info('Mean Absolute Error:', metrics.mean_absolute_error(y_test, y_pred))
logger_.info('Mean Squared Error:', metrics.mean_squared_error(y_test, y_pred))
logger_.info('Root Mean Squared Error:', np.sqrt(metrics.mean_squared_error(y_test, y_pred)))

new_df = pd.DataFrame({'Date': dates_test.flatten(),
                       'Actual Demand': y_test.flatten(),
                       'Predicted Demand': y_pred.flatten()})
with pd.ExcelWriter('new_df.xlsx') as writer:
    new_df.to_excel(writer, sheet_name='Info')
logger_.info("Completed program execution.. ")
return 0

if __name__ == '__main__':
    status = main(sys.argv[1:])
    sys.exit(status)

```